

AI Trip Planner: a One-Person AI Travel Agency

ELEC5620 Model-Based Software Engineering, Stage 1 Report

Requirements documentation, architecture design and system modelling

Member	Name	SID	GitHub	Area
A	Ziqi He	530527086	@Lilstanie	Orchestrator and integration
B	Tingsong Jin	540532755	@fonever2	Itinerary and transport
C	Yi Qiao	550378747	@HeadmasterEggy	Accommodation and budget
D	Jiahang Bian	550066431	@jbia0391	Destination guide and dining
E	Weihao Wang	550414791	@Whw0591	Web workspace and memory

Group: Mon 10-12 Group 14

Repository: https://github.com/Lilstanie/AI_TRIP_PLANNER

Video: *YouTube link*

Contents

1. Project requirements
2. Requirement classification
3. Feature model
4. Use cases
5. Architecture analysis and design
6. Elementary structure: class model
7. Complex structure: object, collaboration and structured class
8. Behaviour models (individual)
9. Relationships between key components and future development
10. Design rationale and discarded choices
11. Change assessment and acceptance
12. Design assumptions
13. Contributions and use of generative AI

1. Project requirements

1.1 Problem

Planning a trip means visiting many sites: one for flights, one for hotels, a map for routes, blogs for food and customs, and a spreadsheet to keep the total inside a budget. Each choice constrains the others. A cheaper flight that lands late cuts the first day short, and an expensive hotel leaves no money for food. General chatbots can write an itinerary, but they invent prices and opening hours, and they never check that the parts fit together.

AI Trip Planner is a **one-person AI travel agency**. One human founder runs the agency, and a team of LLM-based agents does the work a travel agency's staff would do. A traveller describes a trip in chat. The agents then plan it together on a shared planning board, using real map, hotel, flight and weather data, and the system checks their proposals against each other and against the budget before the traveller sees the plan.

1.2 Primary users and roles

Role	Kind	Responsibility
Traveller	Human, primary user	Describes the trip, sets preferences and filters, reads and edits the plan.
Founder / operator	Human	Runs the agency: configures models, data providers and quotas, chooses mock or live data, and uses the Agent Lab to measure and compare the agents before a change ships.
AI Travel Coordinator	AI role	Talks to the traveller, turns the conversation into a structured <code>TripBrief</code> , asks clarifying questions, and explains the result.
AI Planning Supervisor	AI role	Inside the deterministic LangGraph workflow, decides which specialists a planning or revision step needs and what each should focus on.
AI Itinerary Planner	AI specialist	Builds the day-by-day schedule and checks that each day's route is feasible.
AI Transport Agent	AI specialist	Plans flights, inter-city rail and local routes, with times and fares.
AI Accommodation Agent	AI specialist	Finds and compares stays, allocates rooms for individuals or groups, and fits the stay budget.
AI Destination Guide	AI specialist	Gives grounded attractions, customs, safety, entry and packing advice, including weather.
AI Dining Agent	AI specialist	Recommends grounded venues that respect dietary needs and the meal budget.
External services	Systems	Maps and routes (Google, OpenStreetMap), hotel and flight search (SerpApi, Google Places), weather (Google, Open-Meteo), DeepSeek LLM.

The AI roles are clearly different: each has its own system prompt, inputs, tools, output schema and revision behaviour, and they collaborate through the planning board rather than one model answering everything.

1.3 Key capabilities and core features

For the traveller

- *Conversational trip intake.* The traveller writes in English or Chinese; the coordinator extracts only what was said into a `TripBrief` and asks for what is missing instead of guessing.
- *Coordinated multi-agent plan.* Five specialists plan in stages (transport, then accommodation, then itinerary and dining), each within its share of the budget left by earlier stages.
- *Conflict detection and targeted revision.* Budget overruns, schedule overlaps and impossible routes are detected deterministically and sent back only to the agents involved, for at most three rounds.
- *Grounded, labelled data.* Every price and place comes from a tool, and every section says whether its data is live, estimated, mock or a fallback.
- *Editable workspace.* The plan appears as a timeline and a map; edits re-check routes, times and the budget.
- *Memory.* Preferences and chat history persist, and signed-in travellers sync trips across devices.

For the founder / operator

- *Agent Lab.* Runs fixed scenarios through a single-agent baseline, five specialists without revision, and five specialists with targeted revision, and compares deterministic metrics.
- *Failure Lab.* Injects registered faults (provider timeout, empty result, invalid output, supervisor failure, stalled revision) to show the system degrades safely.
- *Mock or live data per request, and provider quotas,* so the agency can control cost.

1.4 Feature ownership: two core and one optional feature per member

Member	Core feature 1	Core feature 2	Optional feature
A	Conversational brief extraction and clarifying questions	Multi-agent orchestration with conflict detection and targeted revision	Agent Lab comparison and Failure Lab
B	Itinerary scheduling with route feasibility	Transport: flights, inter-city rail and local routes	Traveller-chosen leg mode and own flights
C	Accommodation for individuals or groups	Budget management with targeted savings	Keep a stay already booked
D	Destination guide: attractions, customs, safety, entry	Dining with dietary constraints	Weather-based packing advice
E	Chat workspace with timeline and map editing	Preferences and memory	Accounts and cloud sync

Each member: check your row matches what you implement in Stage 2.

2. Requirement classification

2.1 Agreement, mandatory capabilities and optional features

Agreement (scope the group and the stakeholder agreed). The product is a single-user planning workspace: describe a trip, inspect grounded recommendations, edit the plan, save it and reopen it. Prices are estimates in AUD; the system does not book, pay or refund. Multi-user collaboration, social features, in-app payment and booking fulfilment are out of scope for Stage 2. Models may reason and choose, but they never decide the control flow, invent a price or place, or override a confirmed preference; deterministic code checks every model output.

Mandatory capabilities (the product is not acceptable without them):

ID	Capability	Owner
M1	Chat intake that extracts a structured trip brief and asks for missing facts	A
M2	LangGraph planning loop: dispatch, conflict detection, targeted revision (at most 3 rounds), build plan	A
M3	Day-by-day itinerary with route-feasibility checks	B
M4	Transport between and within cities, with times and fares	B
M5	Accommodation for individuals or groups within the stay allocation	C
M6	Budget roll-up in AUD, overrun spreading and infeasible-budget detection	C
M7	Grounded destination guide	D
M8	Dining recommendations that respect dietary needs	D
M9	Workspace with timeline and map views and plan editing	E
M10	Preference and session memory	E
M11	Provenance label on every section; deterministic fallback when a model or provider fails	All

Optional features (add value; the product is acceptable without them): Agent Lab and Failure Lab (A); traveller-chosen leg mode and own flights (B); keep a booked stay (C); weather-based packing advice (D); accounts and cloud sync (E); attachments in chat; place photos.

2.2 Ad hoc requirements (individual)

Informal statements in the stakeholder's own words, before any modelling.

ID	Member	Ad hoc requirement
AH-C1	C (@HeadmasterEggy)	"We're five friends going to Tokyo on a fixed budget. I want the planner to work out how many rooms we need, whether we share or each get our own, and pick a hotel I'd actually stay in: decent rating, free cancellation if I asked for it. Flights get paid first, so the hotel has to fit in whatever money is left. If the whole trip ends up over budget, I want the hotel swapped for a cheaper one that still meets my rules rather than being told nothing fits. If no hotel can ever fit, tell me the minimum I'd need instead of making up a cheap one. And if I've already booked somewhere, just keep it."
AH-A1	A	<i>to be written by A</i>
AH-B1	B	<i>to be written by B</i>

ID	Member	Ad hoc requirement
AH-D1	D	<i>to be written by D</i>
AH-E1	E	<i>to be written by E</i>

Group-level needs gathered in early lab discussion (from the team's design doc): clothing advice from the weather, accommodation for individuals or groups, food recommendations, day-by-day scheduling, luggage rules, transport, a budget, a chat window that splits a request into tasks and merges the answers, and filters for party size, area and budget.

2.3 Classified requirements: member C

AH-C1 is broken down into classified requirements below. FR = functional, NFR = non-functional, C = constraint.

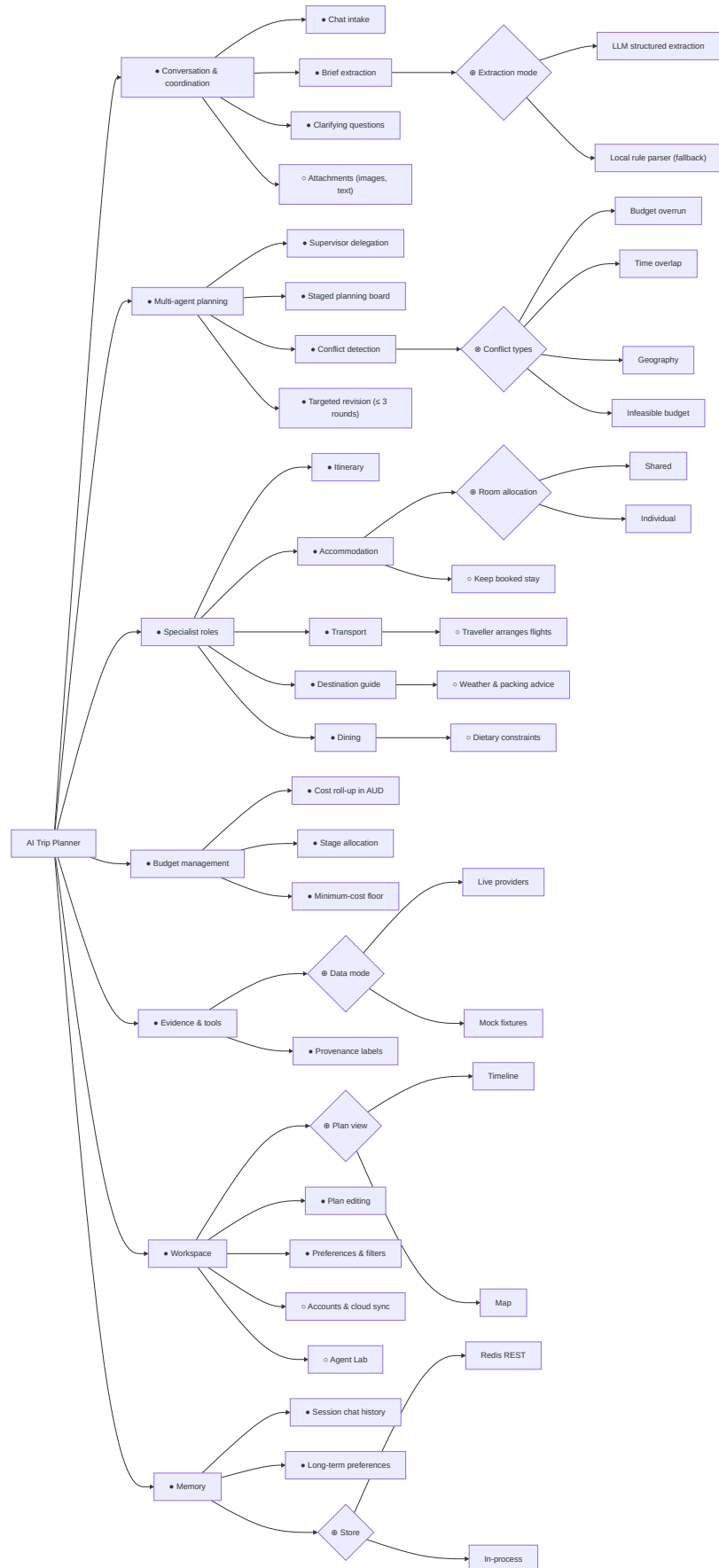
ID	Type	Requirement	Traced to
R-C1	FR	Compute the room count from party size and room allocation: <code>individual</code> gives one room per guest, <code>shared</code> gives <code>[guests / 2]</code> .	<code>accommodation/index.ts</code>
R-C2	FR	Filter stay candidates by minimum rating and, when requested, free cancellation; these are hard rules that a budget revision may not relax.	<code>planning.ts: eligibleOptions</code>
R-C3	FR	Choose the first stay preferring rating ≥ 8 and free cancellation, within the stay allocation left after transport.	<code>chooseInitial</code> , board allocation
R-C4	FR	Roll up section costs in AUD and compare the total with <code>budgetTotal</code> .	<code>budget.ts: rollUpCost</code>
R-C5	FR	On any overrun, spread the required saving over the sections that can still be cut, and send each a targeted revision request.	<code>conflicts.ts: detectConflicts</code>
R-C6	FR	When the cheapest options already exceed the budget, report one <code>infeasible budget</code> conflict naming the minimum, and stop revising.	<code>minimumCost</code> , <code>INFEASIBLE_BUDGET</code>
R-C7	FR	Keep a stay the traveller has already booked, unpriced and without a search.	<code>bookedStayProposal</code>
R-C8	NFR (accuracy)	Money is summed in integer cents so totals never drift.	<code>sumMoney</code> , <code>stayCost</code>
R-C9	NFR (integrity)	The model may choose only among searched candidate ids; it never invents a property, rate or policy.	accommodation system prompt
R-C10	NFR (availability)	With no model key, or an off-schema answer, a deterministic fallback still produces a valid proposal.	<code>planStays</code> fallback
R-C11	C	At most three revision rounds; a round is kept only if the plan score improves.	<code>workflow.ts</code>

Classified requirements for members A, B, D and E. Each member adds a table in the same format (FR, NFR, C) for their own ad hoc requirement.

3. Feature model

3.1 Feature diagram (group)

Notation: • mandatory, ○ optional, $\oplus$ alternative (exactly one), $\otimes$ or (one or more).



3.2 Cross-tree constraints

1. *Targeted revision requires Conflict detection.*

2. *Budget overrun* requires *Cost roll-up in AUD*; *Infeasible budget* requires *Minimum-cost floor*.
3. *Keep booked stay* excludes searching and pricing in *Accommodation* for that trip.
4. *Traveller arranges flights* excludes fare search in *Transport* and any flight-fare conflict.
5. *Live providers* requires provider keys; without them the system selects *Mock fixtures*.
6. *Accounts & cloud sync* requires *Long-term preferences*.

3.3 Non-functional requirements attached to features

Feature	NFR
Multi-agent planning	Every proposal is re-validated against the shared Zod schema at each graph boundary.
Specialist roles	Each specialist falls back to deterministic output when the model is missing or off-schema, so a request always completes.
Budget management	AUD arithmetic in integer cents; overrun percentages are compared unrounded.
Evidence & tools	Prices and places come only from tools; every section shows whether its data is live, estimated, mock or fallback.
Workspace	Planning progress streams as NDJSON so the traveller sees each stage as it runs.
Agent Lab	Fixture runs can never reach a paid model or provider.

4. Use cases

4.1 Overall use case diagram

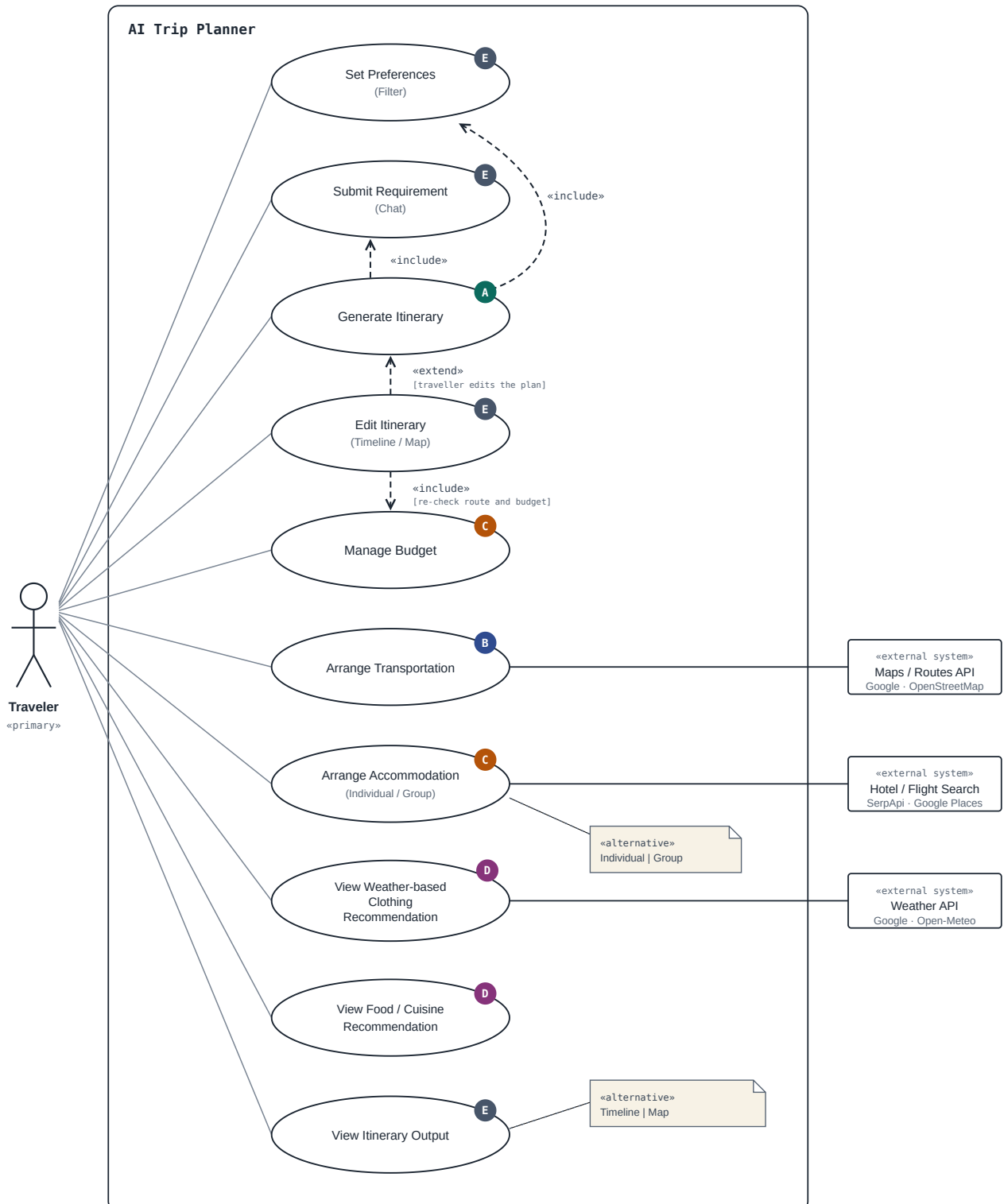


Figure 4.1 Overall use case diagram. Actors: **Traveler** (primary), and three external systems: **Maps / Routes API** (Google, OpenStreetMap), **Hotel / Flight Search** (SerpApi, Google Places) and **Weather API**

(Google, Open-Meteo).

Use case	Owner	Relationships
Set Preferences (Filter)	E	—
Submit Requirement (Chat)	E	—
Generate Itinerary	A	«include» Arrange Transportation, Arrange Accommodation, Manage Budget
Edit Itinerary (Timeline / Map)	E	«extend» Generate Itinerary [traveller edits the plan]
Arrange Transportation	B	«include» Maps / Routes API
Arrange Accommodation (Individual / Group)	C	«include» Hotel / Flight Search
Manage Budget	C	«extend» Edit Itinerary [re-check route and budget]
View Weather-based Clothing Recommendation	D	«include» Weather API
View Food / Cuisine Recommendation	D	—
View Itinerary Output	E	«alternative» Timeline / Map

4.2 Use case specifications: member C

UC-C1 Arrange Accommodation

Field	Content
ID / name	UC-C1 Arrange Accommodation (Individual / Group)
Source	AH-C1; R-C1, R-C2, R-C3, R-C7, R-C9, R-C10
Primary actor	Traveler
Secondary actors	Hotel / Flight Search (SerpApi Google Hotels, Google Places), Orchestrator (system role)
Goal	Every night of the trip has a stay that respects the traveller's room and quality rules and fits the money left after transport.
Trigger	The Orchestrator's <code>dispatch_specialists</code> node delegates to the Accommodation agent once Transport has finished on the planning board.
Preconditions	1. A valid <code>TripBrief</code> exists (destination, dates with at least one night per city, <code>groupSize</code> ≥ 1 , <code>budgetTotal</code>). 2. Transport's proposal is on the board, so a stay allocation can be computed.
Postconditions (success)	An <code>AgentProposal</code> for accommodation is on the board with one <code>StaySelection</code> per city, its cost, a <code>floorCost</code> (cheapest eligible total) and a provenance label.
Postconditions (failure)	No proposal; the run stops with the accommodation specialist named as the failure. No stay is invented.

Main success scenario

1. Orchestrator invokes the Accommodation agent with the brief, the board and a stay allocation (40% of the budget left after transport).
2. Agent reads the traveller's lodging preferences: room allocation, minimum rating, free cancellation.
3. Agent computes the room count: one per guest for *individual*, $\lceil \text{guests} / 2 \rceil$ for *shared*.
4. Agent splits the trip into stay segments per city, using the inter-city hops Transport scheduled.
5. For each segment, agent searches the booking port for stays in that city and dates.

6. Agent removes malformed and preference-incompatible candidates and sorts the rest cheapest first.
7. The model chooses one candidate id per segment, weighing rating, cancellation and total cost against the allocation.
8. Agent validates the choice against the candidate list and prices it as $\text{rooms} \times \text{nights} \times \text{nightly rate}$, in cents.
9. Agent records the cheapest eligible total as `floorCost` and returns the proposal to the board.
10. Orchestrator includes the accommodation section in conflict detection (UC-C2).

Extensions

- 2a. *Traveller has already booked a stay* (`bookedStay` set): agent returns that stay unpriced with `floorCost` 0 and skips steps 3–9.
- 4a. *Transport scheduled no usable hops*: nights are split evenly across cities, extra nights to earlier cities.
- 5a. *Live search unavailable*: Google Places estimates are used and the proposal is labelled "estimated", naming the provider that failed.
- 6a. *No candidate survives the filters*: the agent raises an error and the run stops; it does not relax a confirmed preference.
- 7a. *No model configured, or the model returns an unknown id or off-schema output*: the deterministic rule picks the cheapest candidate with rating ≥ 8 and free cancellation (or the cheapest) within the allocation, labelled "Local fallback".
- 7b. *First choice exceeds the allocation*: the best stay within it is taken, or the cheapest if none fit, and the proposal says why.
- *Any step. Revision request received* (from UC-C2): see UC-C2 step 6.

Special requirements: R-C8 (cent arithmetic), R-C9 (no invented properties), R-C10 (fallback).

UC-C2 Manage Budget

Field	Content
ID / name	UC-C2 Manage Budget
Source	AH-C1; R-C4, R-C5, R-C6, R-C11
Primary actor	Traveler
Secondary actors	Orchestrator, the five specialist agents
Goal	The plan the traveller receives fits the budget, or says plainly by how much it cannot.
Trigger	All dispatched specialists have put a proposal on the board (<code>detect_conflicts</code>), or the traveller edits the plan.
Preconditions	<code>budgetTotal</code> $\geq$ AUD 0.01; at least one proposal exists.
Postconditions (success)	<code>TripPlan.estTotal</code> $\leq$ <code>budgetTotal</code> ; no budget conflict remains.
Postconditions (partial)	The plan is returned with the best total found; each section still targeted is marked <code>needs_you</code> , and <code>conflicts</code> lists what remains.

Main success scenario

1. Orchestrator rolls up every proposal's item costs in AUD cents and computes `overrunPct`.
2. Total is within budget: no budget conflict is raised.

3. Orchestrator builds the plan, marks each section `draft` and returns it with `estTotal` and `overrunPct`.

Extensions

- 2a. *Total over budget, but the sum of floor costs fits:*
 1. Orchestrator computes each section's room to cut (its cost less its `floorCost`).
 2. It spreads the overrun over those sections in proportion, creating one `RevisionRequest` per section with a `targetSaving`.
 3. Orchestrator runs `revise_conflicts` for those agents only, giving each its previous proposal and a new allocation (last cost less saving).
 4. Accommodation, on a budget revision, keeps the confirmed preferences and picks the cheapest eligible stay within the new allocation.
 5. Orchestrator re-runs detection and computes the plan score (AUD over budget + 10% of budget per other conflict).
 6. Score improved: keep the round and go back to step 1 of the main scenario. Score not improved: discard the round and go to 2c.
- 2b. *Sum of floor costs already exceeds the budget:* Orchestrator raises one `infeasible budget` conflict naming the minimum, stops revising, and the reply tells the traveller to raise the budget to at least that amount or change dates, origin or destination.
- 2c. *Three rounds reached, or no improvement:* the best plan so far is returned; sections still targeted are `needs_you`.
- *Traveller edits the plan:* costs are re-checked in the editor and a `price_unverified` issue is shown for any unpriced change.

Special requirements: R-C8; overrun percentages are compared unrounded.

4.3 Use case specifications: members A, B, D and E

Each member adds at least one specification in the format of UC-C1 (the group needs 5 to 10 in total). Suggested: A, Generate Itinerary; B, Arrange Transportation; D, View Weather-based Clothing Recommendation; E, Edit Itinerary (Timeline / Map).

5. Architecture analysis and design

5.1 Viewpoints

The architecture is described from four viewpoints, chosen for the concerns of each stakeholder.

Viewpoint	Stakeholder and concern	Where it is modelled
Logical	Developers: which classes and contracts exist, and how they relate	Class model (§6), feature model (§3)
Process	Founder and developers: how agents run, wait for each other, retry and stop	Ad hoc design diagram (§5.2), behaviour models (§8), collaboration (§7.2)
Development	The five members: who owns which package, and which dependencies are allowed	Package diagram (§5.3)
Physical	Founder: where the system runs and which external services it pays for	Deployment diagram (§5.4)
Scenarios	Traveller: what the system does for them	Use cases (§4)

5.2 Ad hoc design diagram: runtime flow

One request from the traveller's message to a validated plan. Solid arrows are the normal path; dashed arrows are fallbacks when a model or provider is unavailable.

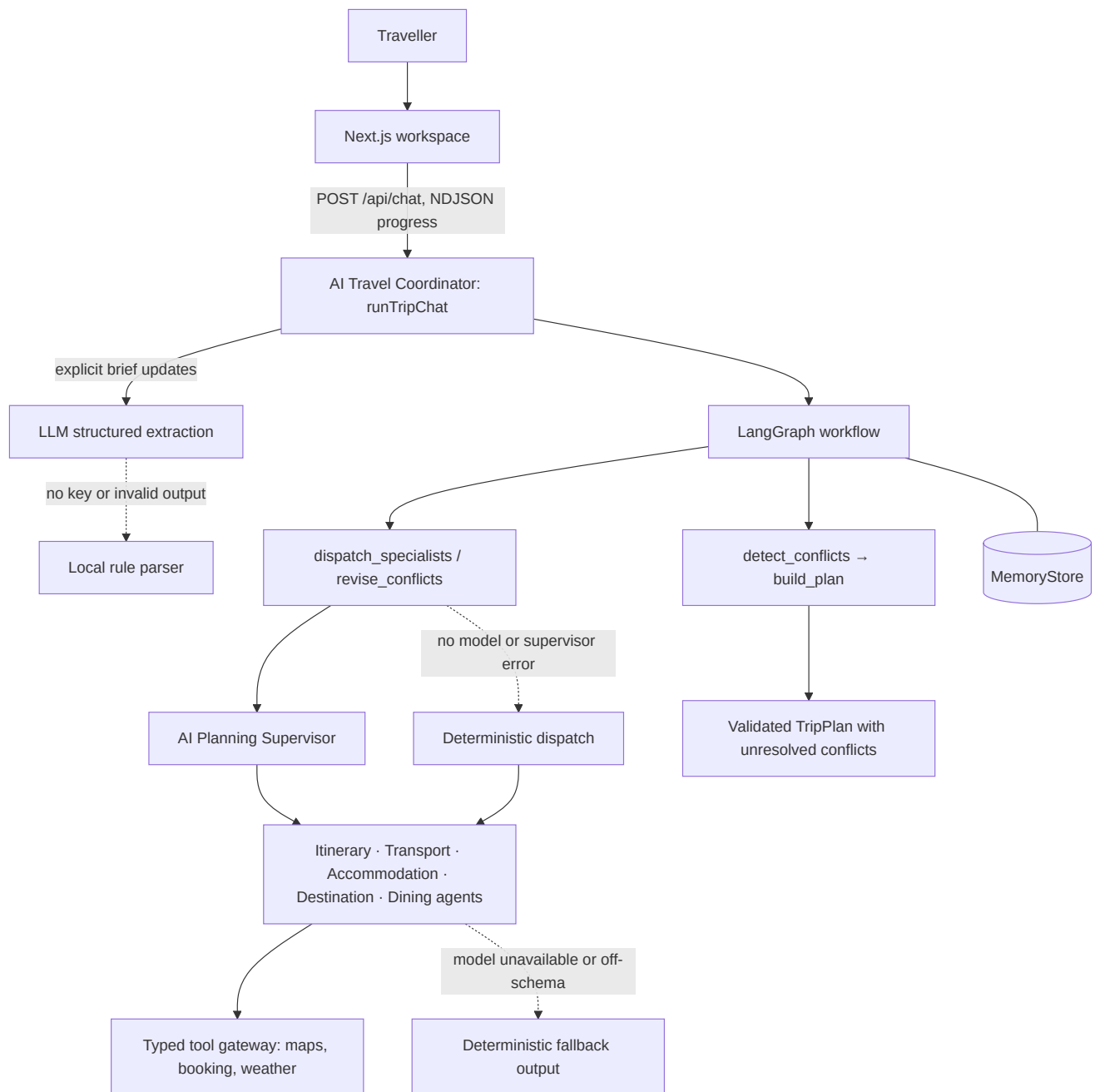


Figure 5.1. Runtime flow.

The LangGraph workflow drives the supervisor, not the other way round. The graph decides when to dispatch, revise and stop; the supervisor only chooses which specialists a step needs. The control path, budget red lines and stopping condition never depend on a model improvising the next step.

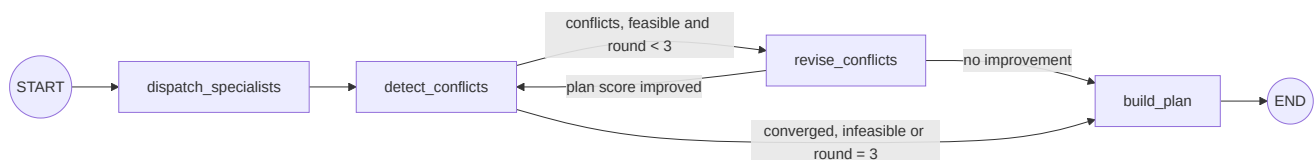


Figure 5.2. The LangGraph planning loop.

5.3 Package diagram

The repository is a pnpm and Turborepo monorepo. Dependencies point inward to the shared contracts; `shared` depends on nothing, and no package imports the web app.

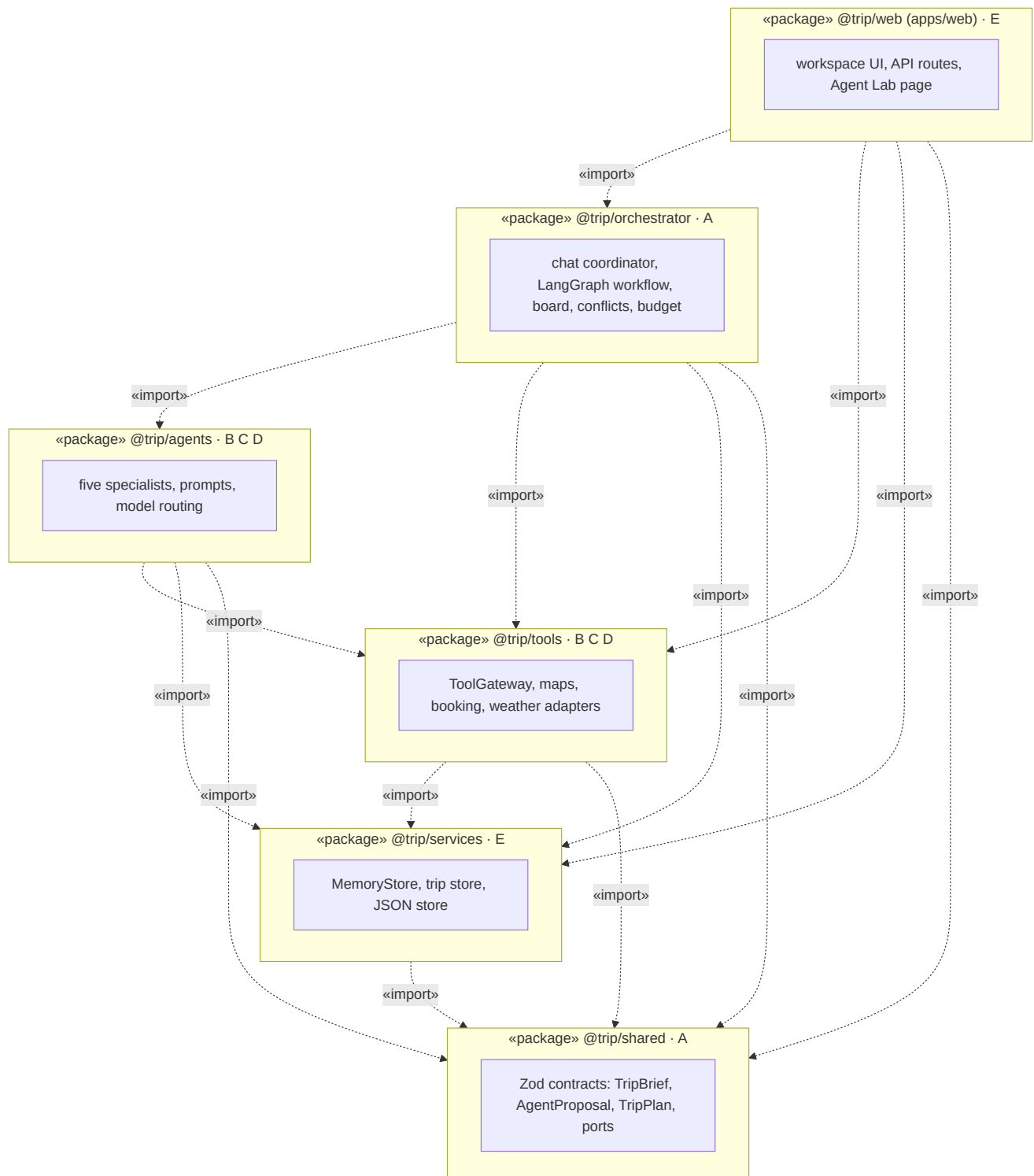


Figure 5.3. Package diagram, with the owning member of each package.

5.4 Deployment diagram

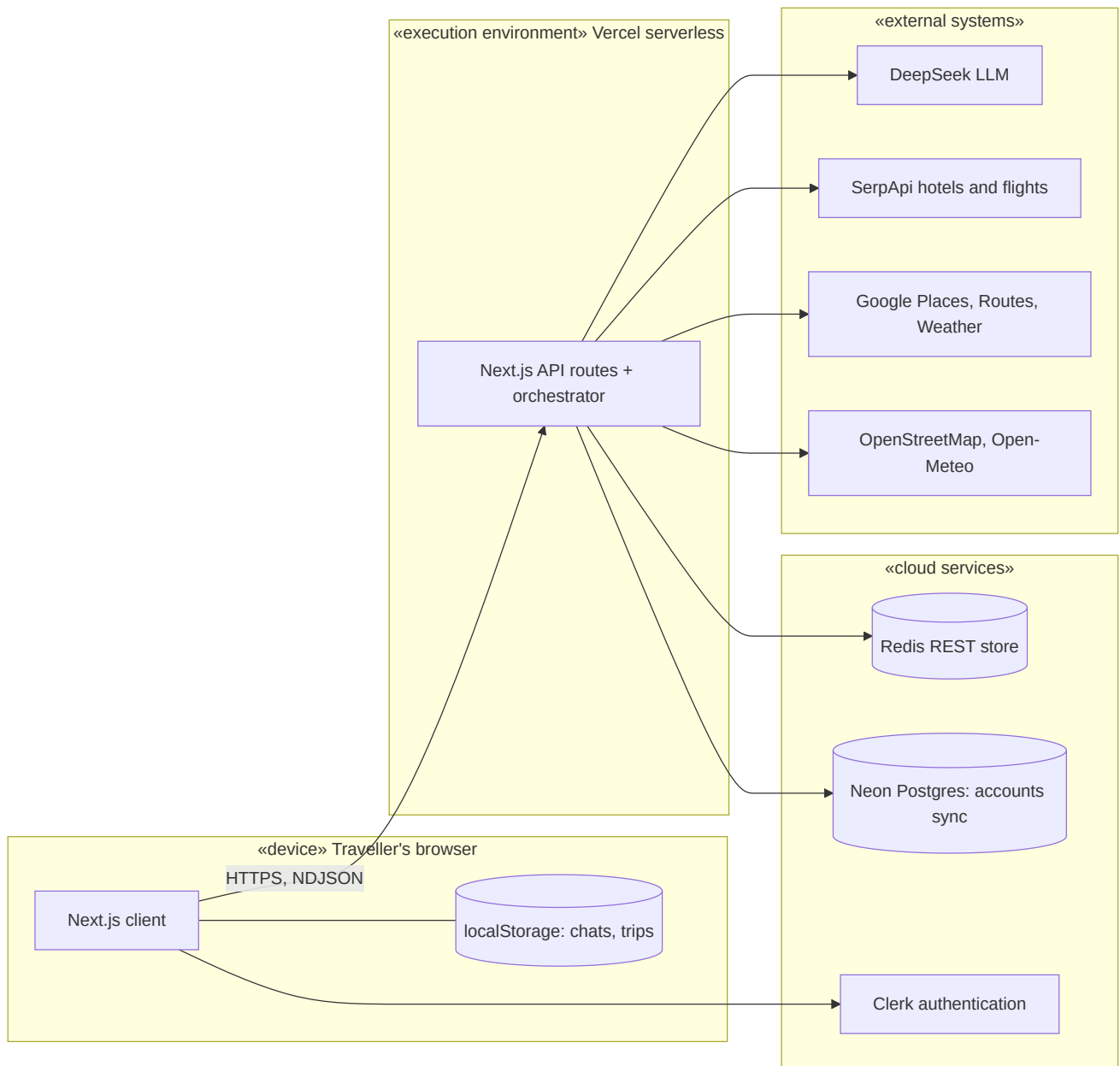


Figure 5.4. Deployment diagram.

Without provider keys the same build runs fully offline on mock fixtures and process memory, which is how the Agent Lab and the tests run.

5.5 Life-cycle process

The group follows an iterative, agile process. Work is tracked as GitHub issues with triage labels and will be mirrored in Jira for Stage 2, as the brief requires. Each change is a short-lived branch merged into `main` through a pull request after CI (typecheck, lint, tests, build, documentation and protected-file checks). Decisions with lasting rationale are recorded as Agent Notes in the repository, so the design model and the code are reviewed in the same pull request.

6. Elementary structure: class model

The class model has six UML class diagrams that share one namespace, drawn from the code. Modules that export functions appear as classes with the `«module»` stereotype. Diagrams 1 to 5 show composition, aggregation, association, dependency, multiplicity, interfaces and realisation; Diagram 6 adds generalisation.

6.1 Notation

Mark	Relationship	Meaning
solid line, hollow triangle	generalisation	subclass → superclass ("is a kind of")
dashed line, hollow triangle	realisation	class → interface ("implements")
solid line, filled diamond	composition	whole ◆ part; part cannot outlive the whole
solid line, hollow diamond	aggregation	whole ◇ part, shared; part has an independent lifetime
solid line, open arrow	association	source holds a stored reference to the target
dashed line, open arrow	dependency	transient use only (parameter, return, local); no stored field

6.2 Diagram 1: structural spine

The request path from the web workspace through the chat coordinator and LangGraph workflow to the specialists, tools and stores.

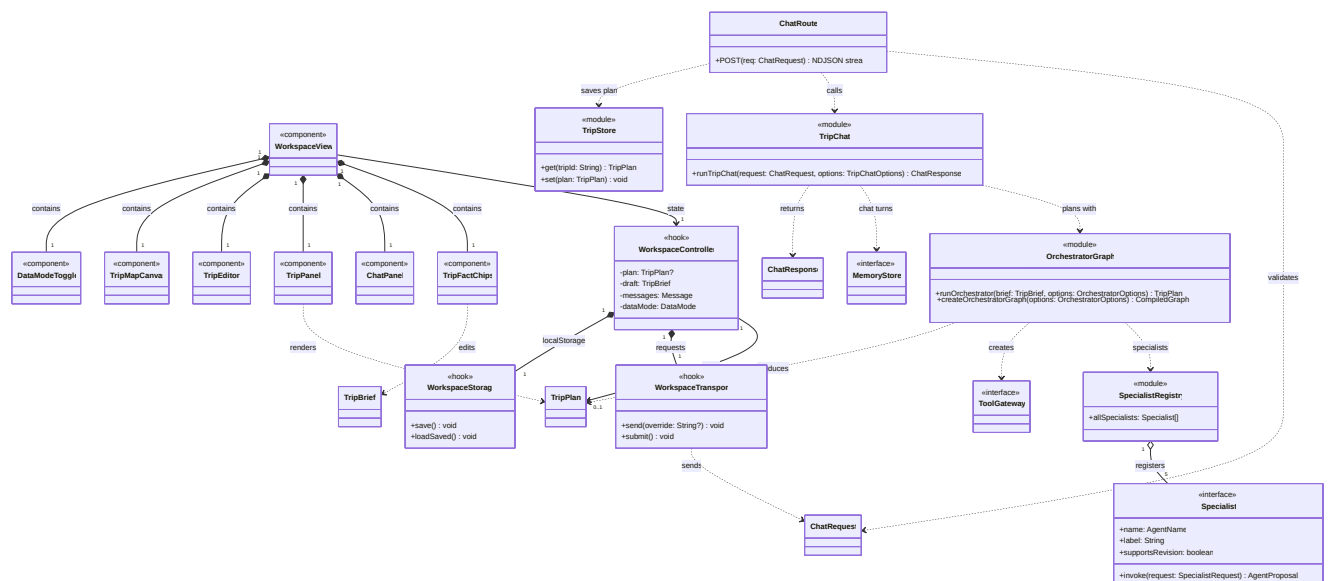


Figure 6.1. Diagram 1: structural spine.

6.3 Diagram 2: domain model

The shared contracts: `TripBrief`, `AgentProposal`, `ProposalItem`, `RevisionRequest`, `TripPlan` and `TripSection`.

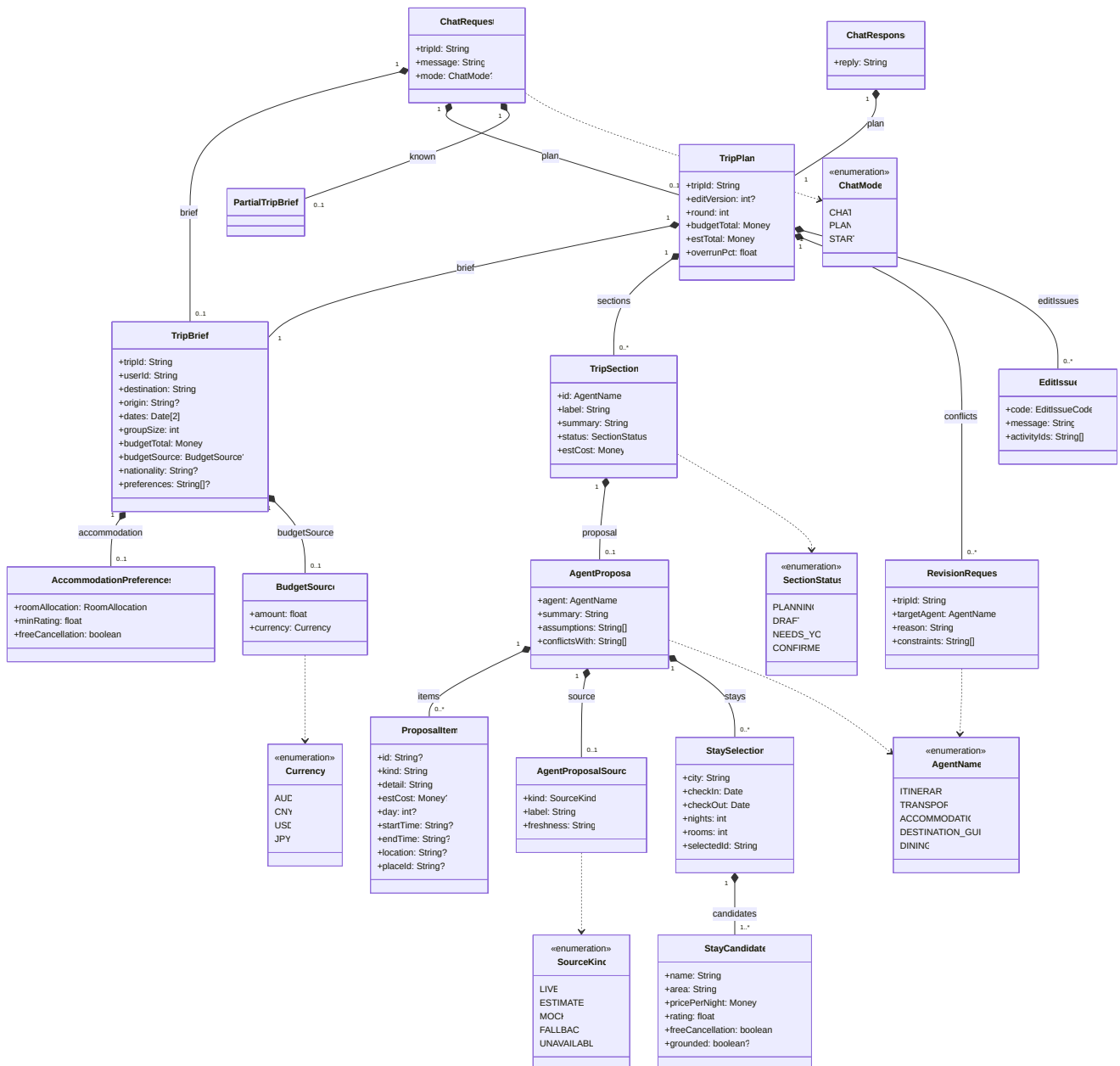


Figure 6.2. Diagram 2: domain model.

6.4 Diagram 3: specialists and orchestration

The `Specialist` interface, its five realisations, the registry and the orchestrator graph.

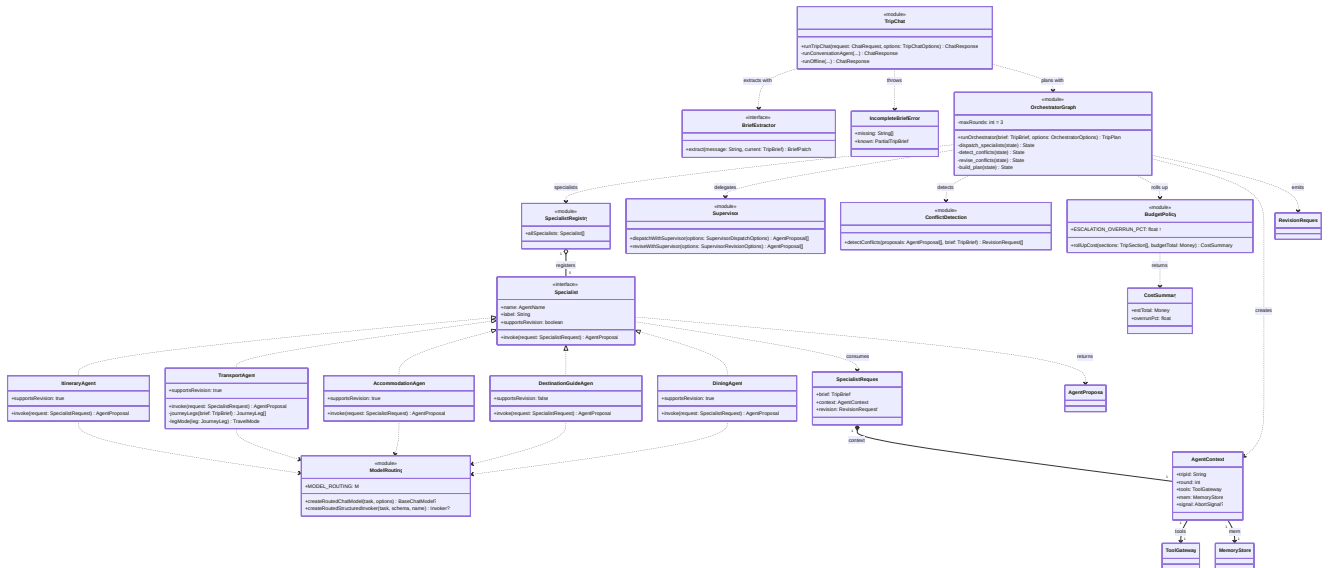


Figure 6.3. Diagram 3: specialists and orchestration.

6.5 Diagram 4: ports, adapters and infrastructure

Hexagonal ports in `packages/shared` and the adapters that realise them.

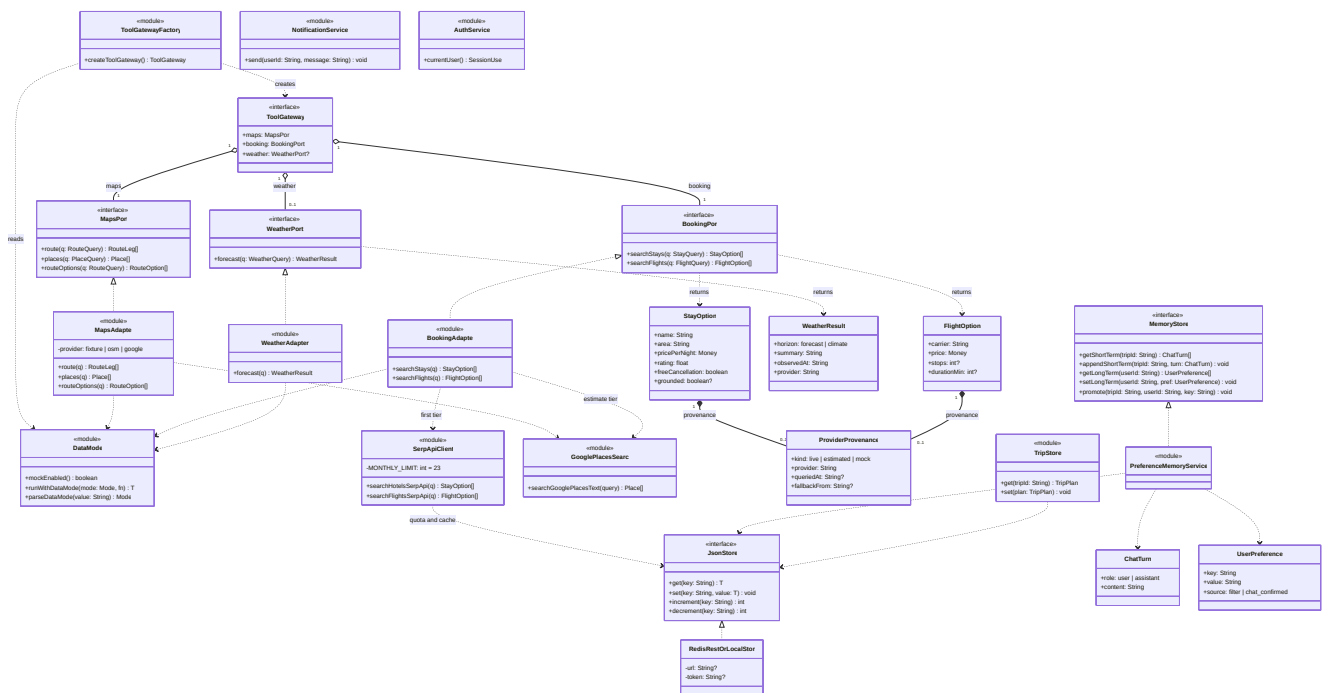


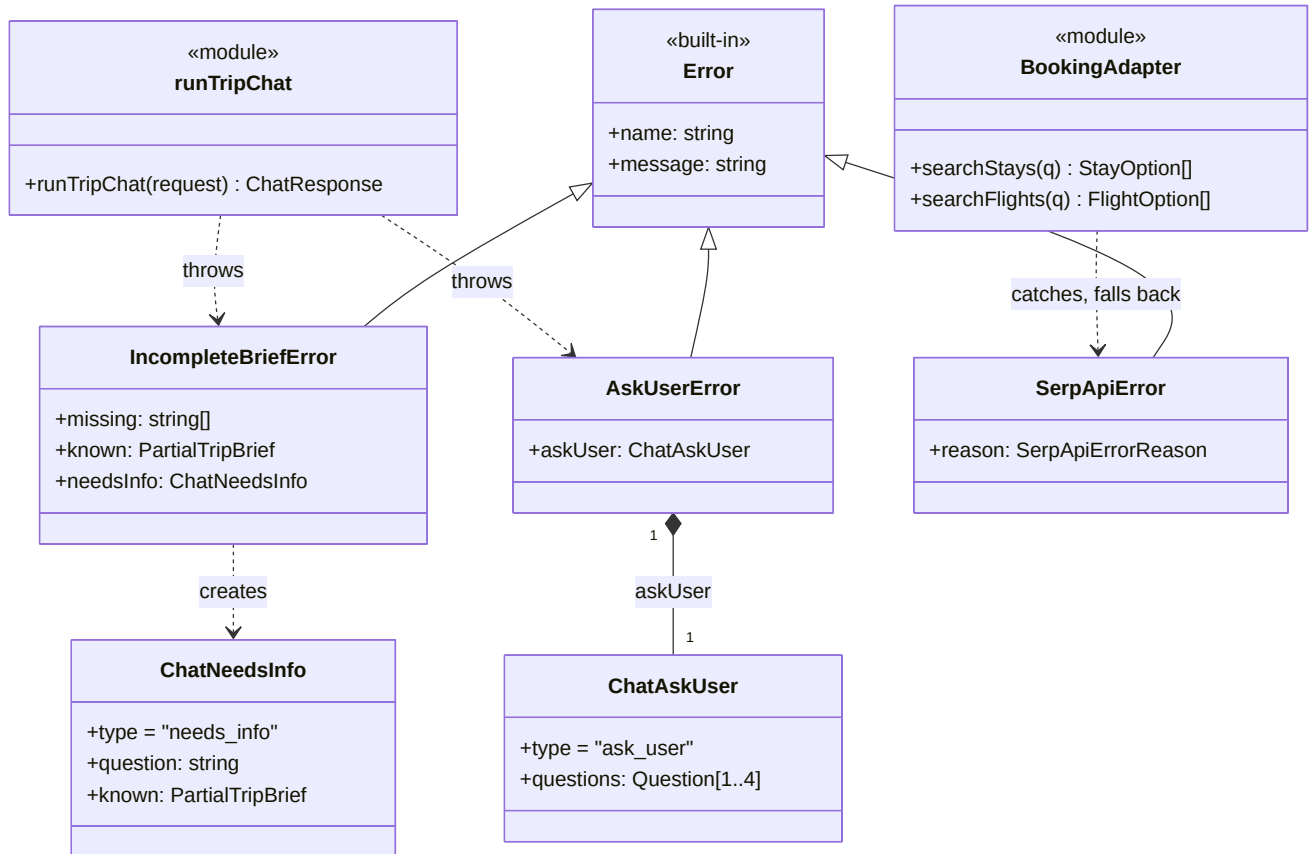
Figure 6.4. Diagram 4: ports, adapters and infrastructure.

6.6 Diagram 5: use cases traced onto the model

Which classes realise each use case.

6.7 Diagram 6: generalisation

The five class diagrams in above already show composition, aggregation, multiplicity, interfaces and realisation (`Specialist <|-- ItineraryAgent`, `MapsPort <|-- MapsAdapter`, ...). The criteria also name **generalisation**, which the existing diagrams do not draw. The code does have real inheritance: the typed errors that turn a failed or unfinished turn into a specific response frame. This is **Diagram 6** of the class model.



Relationship	Kind	Rationale
Error < -- IncompleteBriefError	generalisation	"Not enough to plan yet" is a non-plan outcome, not a crash; the API route catches it by type and streams a <code>needs_info</code> frame with the fields understood so far.
Error < -- AskUserError	generalisation	A clarifying question with 1–4 concrete choices, streamed as <code>ask_user</code> .
Error < -- SerpApiError	generalisation	A provider failure with a typed <code>reason</code> (quota, no results, ...) so the booking adapter can fall back to Google Places estimates and label the provenance.

Design note. Subclassing `Error` lets `runTripChat` end a turn early from deep inside the `LangGraph` or `LangChain` call stack, and lets the route handler distinguish outcomes with `instanceof` instead of string matching.

6.8 Key associations and multiplicity

Read *source* → *target*.

Source	Target	Type	Mult.	Meaning
WorkspaceView	panels, editor, map, data-mode toggle	composition	1 → 1	the workspace renders one of each view
WorkspaceController	TripPlan	association	1 → 0..1	holds the current plan; none before the first plan
WorkspaceController	WorkspaceTransport / WorkspaceStorage	composition	1 → 1	request and localStorage hooks used only by this controller
ChatRoute	TripChat, TripStore	dependency	—	handles one request, then saves the resulting plan
ChatResponse	TripPlan	composition	1 → 1	a response carries a full plan
ChatRequest	TripBrief / TripPlan / PartialTripBrief	composition	1 → 0..1	the browser sends its current state with each message
OrchestratorGraph	Supervisor, ConflictDetection, BudgetPolicy	dependency	—	functions called from graph nodes
SpecialistRegistry	Specialist	aggregation	1 → 5	references module-level agents; no lifecycle ownership
SpecialistRequest	AgentContext	composition	1 → 1	each call carries its own context
AgentContext	ToolGateway / MemoryStore	association	1 → 1	injected — the specialist's only route to tools and memory
TripPlan	TripBrief	composition	1 → 1	embeds the brief it answers
TripPlan	TripSection	composition	1 → 0..*	one section per specialist that returned a proposal
TripPlan	RevisionRequest	composition	1 → 0..*	conflicts still unresolved after the last round
TripSection	AgentProposal	composition	1 → 0..1	the proposal the section was built from, for drill-down
AgentProposal	ProposalItem	composition	1 → 0..*	a proposal is its list of line items
AgentProposal	AgentProposalSource	composition	1 → 0..1	where the data came from: live, estimated, mock, fallback, unavailable
StaySelection	StayCandidate	composition	1 → 1..*	the chosen stay and its alternatives
ToolGateway	MapsPort / BookingPort / WeatherPort	aggregation	1 → 1, 1, 0..1	one port of each kind; weather is optional
StayOption / FlightOption	ProviderProvenance	composition	1 → 0..1	which provider answered and whether it was a fallback

Source	Target	Type	Mult.	Meaning
PreferenceMemoryService / TripStore	JsonStore	dependency	—	every read and write goes through the store

6.9 Interfaces and realisation

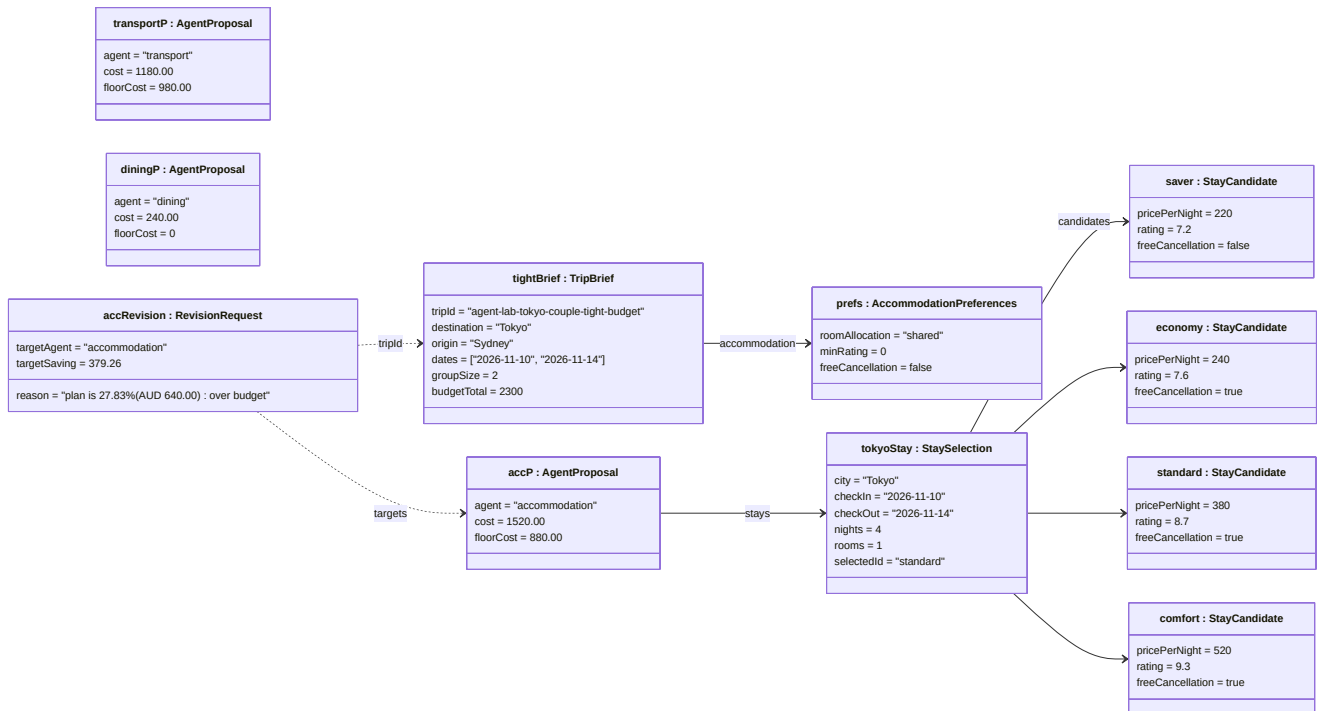
Interface	Realised by	Note
Specialist	ItineraryAgent, TransportAgent, AccommodationAgent, DestinationGuideAgent, DiningAgent	one <code>invoke(SpecialistRequest)</code> ; only the destination guide cannot revise
ToolGateway	object returned by <code>createToolGateway()</code>	one per planning run
MapsPort	MapsAdapter (maps.ts)	owner B; fixtures, OpenStreetMap or Google
BookingPort	BookingAdapter (booking.ts)	owner C; SerpApi, then Google Places estimate, or fixtures; no payment
WeatherPort	WeatherAdapter (weather.ts)	owner D; Google Weather, Open-Meteo or climate context
MemoryStore	PreferenceMemoryService (memory)	owner E
JsonStore	RedisRestOrLocalStore (<code>createJsonStore()</code>)	owner E; Redis REST when configured, process memory otherwise
BriefExtractor	model extractor in chat.ts, or <code>extractBriefPatchLocally</code> without a key	owner A

NotificationService and AuthService are stubs: notifications are logged, and every request is the demo user.

7. Complex structure: object, collaboration and structured class

7.1 Object diagram

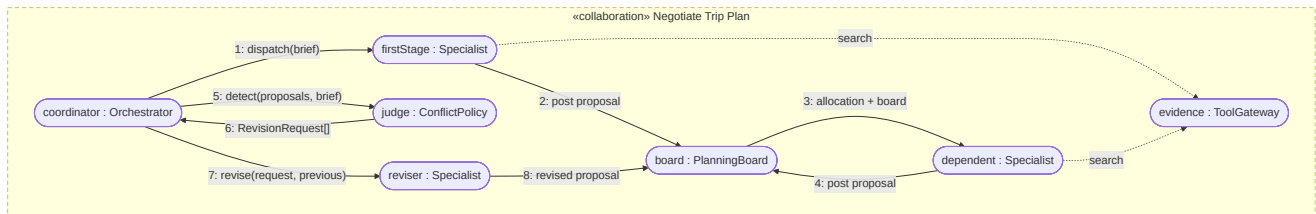
A snapshot of the planning board at the end of round 1 of `detect_conflicts`, for the Agent Lab's `tokyo-couple-tight-budget` brief. The brief and the four stay candidates are the real fixture values (`agent-lab/scenarios.ts`, `tools/src/booking.ts`). **Transport and dining costs are illustrative**: re-run the lab and replace them if you want exact figures.



How to read it: the couple shares one room ($[2 / 2] = 1$). The first choice is *standard*, the cheapest candidate rated ≥ 8 with free cancellation, at $1 \times 4 \times 380 = \text{AUD } 1,520$. The floor is *saver*, at $4 \times 220 = \text{AUD } 880$. The total of AUD 2,940 is AUD 640 (27.83%) over the AUD 2,300 budget. The overrun is spread over each section's room to cut (transport 200, accommodation 640, dining 240, 1,080 in total), so accommodation is asked to save $640 \times 640 / 1,080 = \text{AUD } 379.26$. In round 2 its allocation is AUD 1,140.74. A budget revision takes the cheapest eligible candidate, which is *saver* (AUD 880): free cancellation was not a confirmed preference, so *saver* is still eligible. Had the traveller required free cancellation, *saver* would be filtered out and *economy* (AUD 960) chosen instead.

7.2 Collaboration: «collaboration» Negotiate Trip Plan

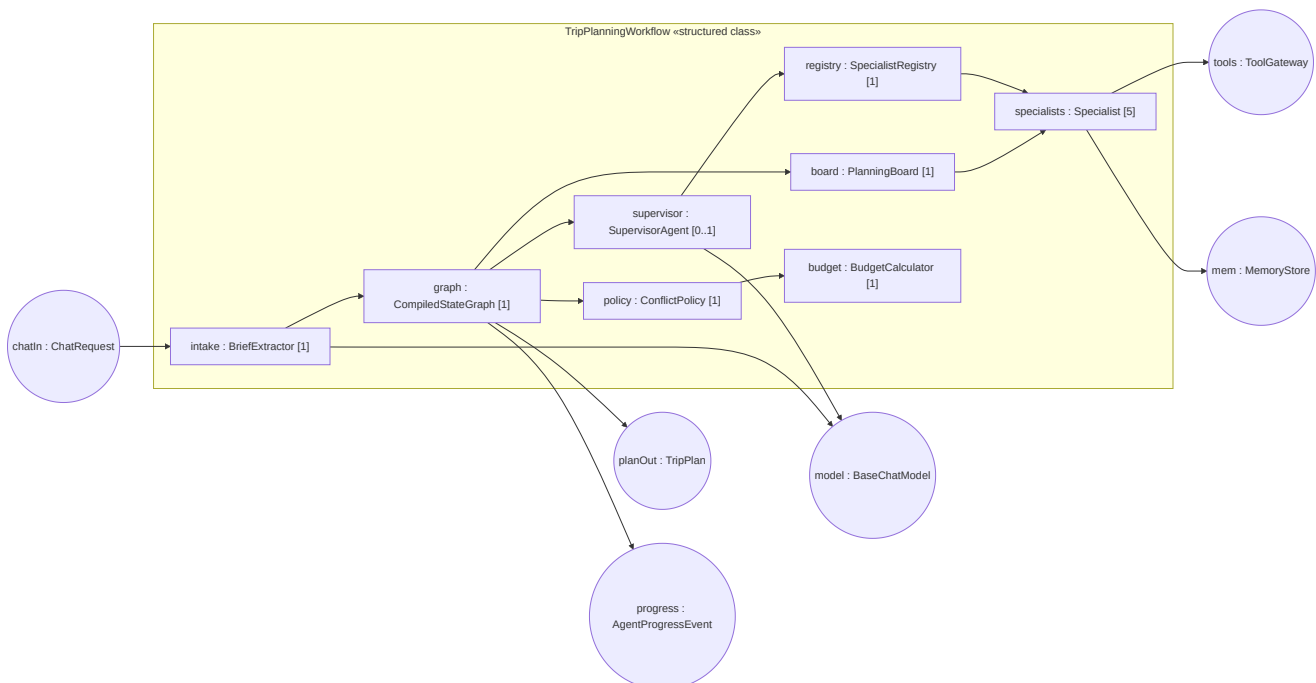
Roles, not classes: any `Specialist` can play a role, and the connectors are the links the roles use during one planning turn.



Role	Played by (in main)	Responsibility in the collaboration
coordinator	LangGraph workflow plus supervisor agent	Decides when to dispatch, detect, revise and stop.
board	<code>board.ts</code>	Orders the stages (transport → accommodation → itinerary and dining) and computes each allocation.
firstStage	Transport (also Destination guide, which has no budget share)	Plans without waiting for anyone.
dependent	Accommodation, Itinerary, Dining	Waits for earlier stages and plans within its allocation.
judge	<code>detectConflicts</code> and <code>rollUpCost</code> (member C)	Turns proposals into targeted <code>RevisionRequest</code> s.
reviser	Any specialist with <code>supportsRevision</code>	Revises only its own section, from its previous proposal.
evidence	<code>ToolGateway</code> (maps, booking, weather)	Supplies grounded places, prices and forecasts.

7.3 Structured class: TripPlanningWorkflow

The internal structure of the orchestrator as a composite: its parts, the ports it exposes and requires, and the connectors between parts.



Element	Kind	Notes
<code>chatIn</code> , <code>planOut</code> , <code>progress</code>	provided ports	<code>POST /api/chat</code> request in; NDJSON progress frames and a final <code>{ reply, plan }</code> out.
<code>tools</code> , <code>mem</code> , <code>model</code>	required ports	Injected through <code>OrchestratorOptions</code> and <code>AgentContext</code> ; tests pass fakes.

Element	Kind	Notes
supervisor [0..1]	part	Absent when no model is configured; the graph then dispatches deterministically.
specialists [5]	part	Itinerary, Transport, Accommodation, Destination guide, Dining.
policy, budget	parts	Member C's detectConflicts, rollUpCost and minimumCost.

8. Behaviour models (individual)

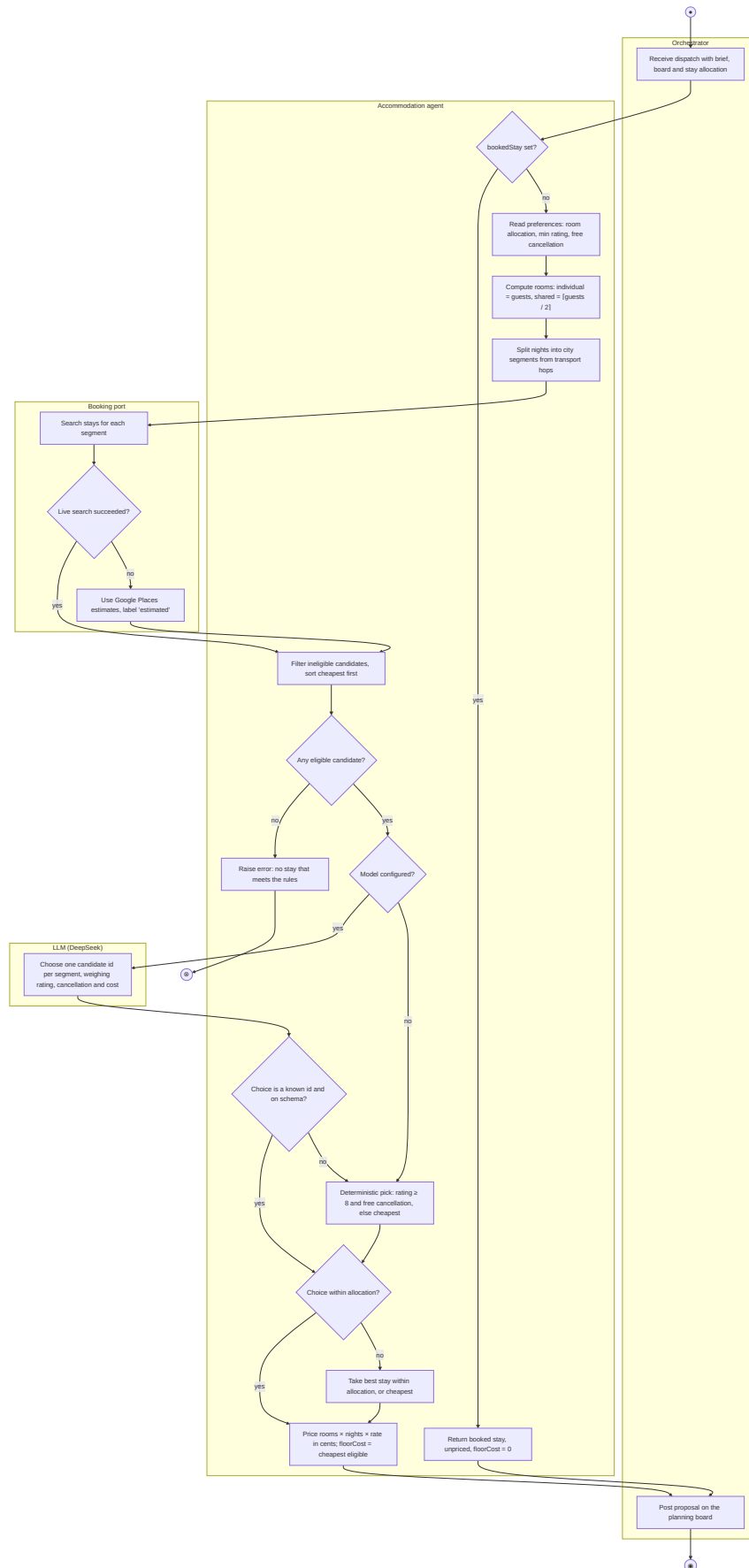
Each member's activity, sequence and state machine diagrams come from their own ad hoc requirement and use case specification, and show where the LLM sits and which deterministic code checks it.

8.1 Member C (@HeadmasterEggy, accommodation and budget)

All three diagrams come from ad hoc requirement **AH-C1** and the use case specifications **UC-C1 Arrange Accommodation** and **UC-C2 Manage Budget** (01-requirements.md, 02-use-cases.md). Each focuses on where the LLM sits and what deterministic code guards it, as the brief asks.

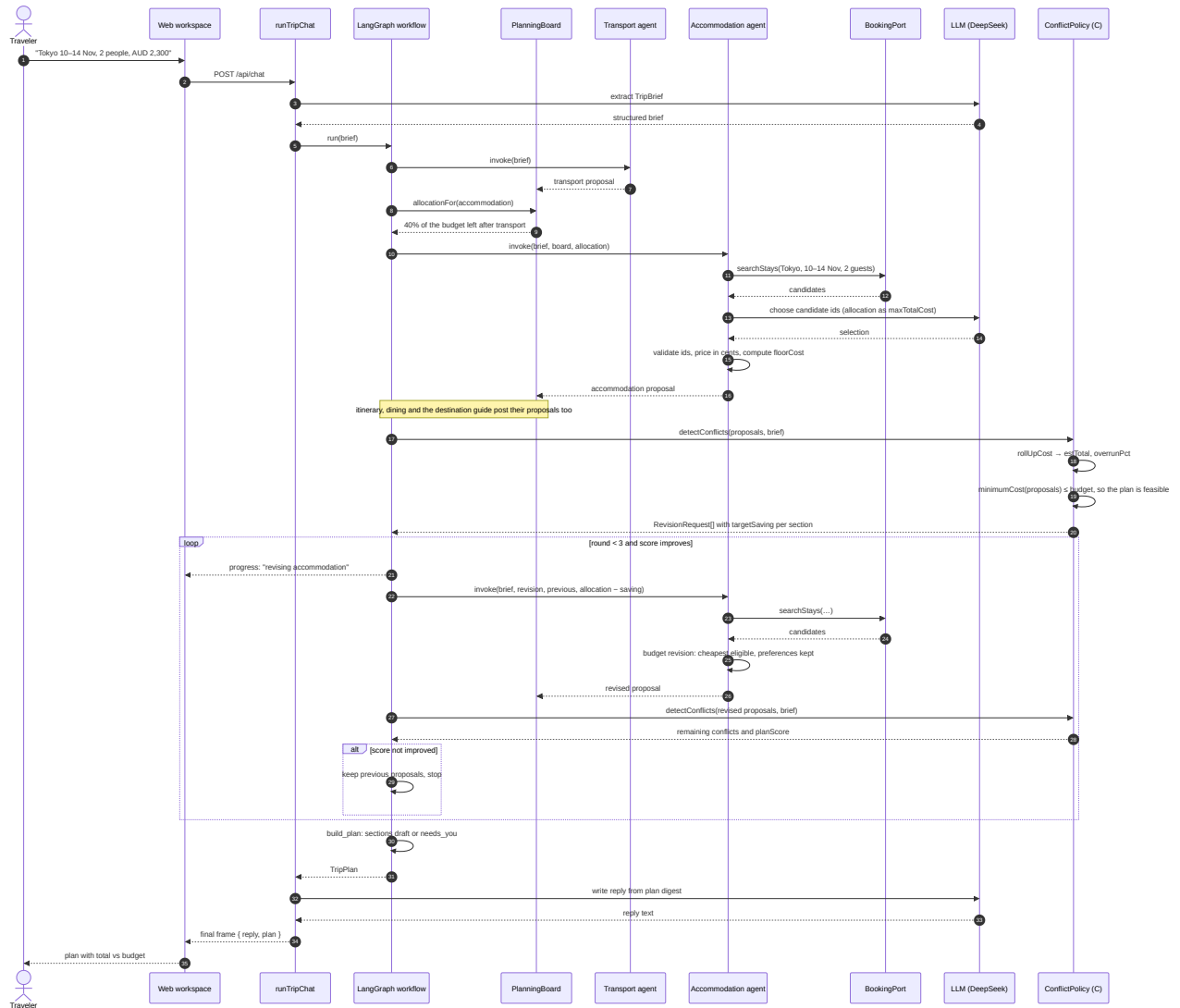
8.1.1 Activity diagram: Arrange Accommodation (UC-C1)

Swimlanes are the four participants. The LLM is used once, in *Choose one candidate id per segment*, and its output is checked before anything is priced.



8.1.2 Sequence diagram: budget overrun and targeted revision (UC-C2, extension 2a)

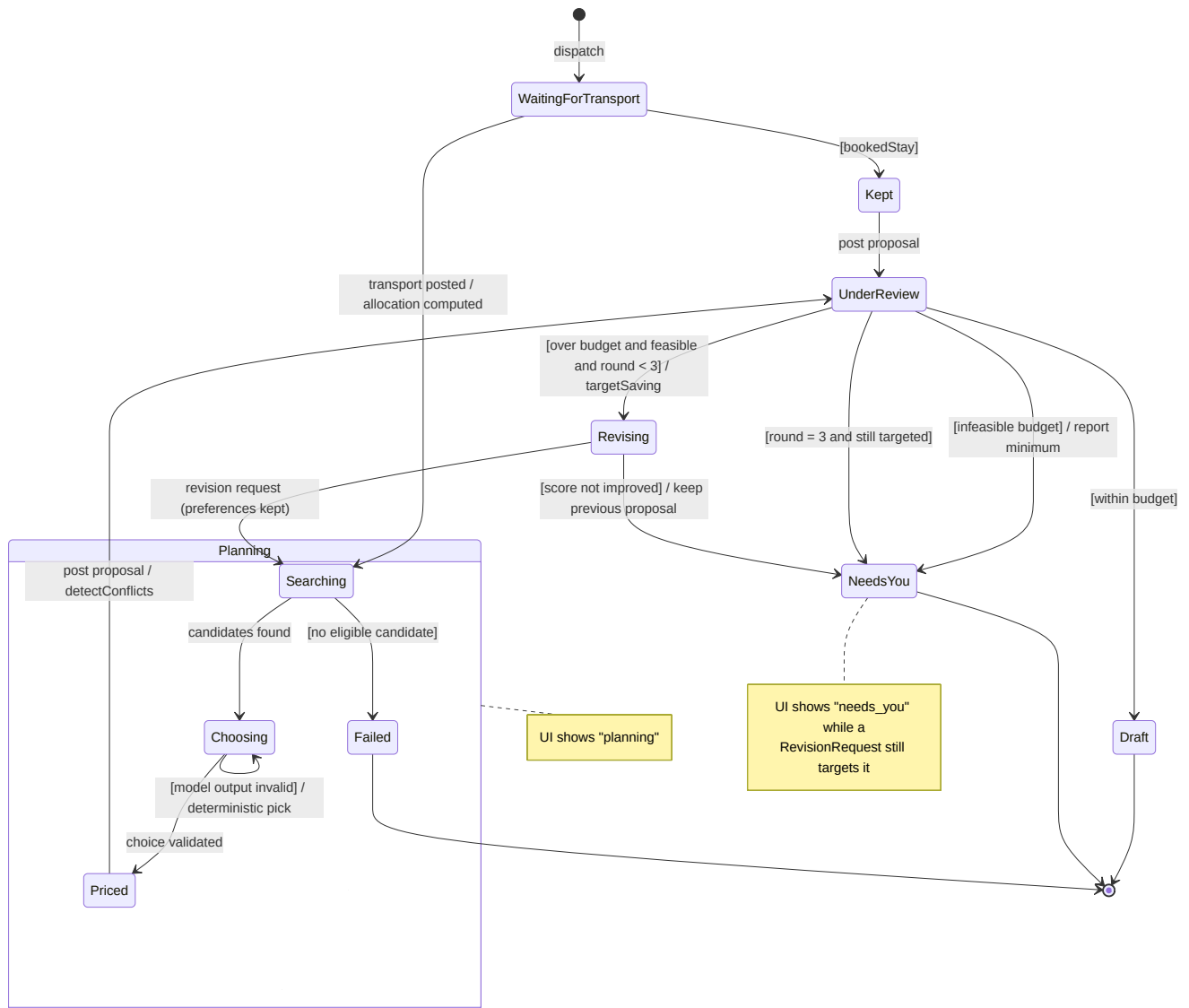
One planning turn in which the first round goes over budget and the accommodation agent is asked to cut its cost.



If `minimumCost` is above the budget (extension 2b), `detectConflicts` instead returns one `infeasible budget` conflict, the loop is skipped, and the reply names the minimum budget needed.

8.1.3 State machine: accommodation section within a planning turn (UC-C1 and UC-C2)

The section's visible status (`planning`, `draft`, `needs_you`) is derived from these internal states.



State	UI status	Entry / exit behaviour
WaitingForTransport	planning	Waits only if transport was started in the same turn.
Searching, Choosing, Priced	planning	The LLM acts only in Choosing; invalid output loops back through the deterministic pick.
Kept	planning	Booked stay, no search; floorCost 0 so it is never asked to cut.
UnderReview	planning	Member C's detectConflicts decides the next state.
Revising	planning	Same preferences, new allocation; the round is kept only if planScore improves.
Draft	draft	No revision request targets the section.
NeedsYou	needs_you	The traveller decides: raise the budget, change dates or edit the plan.
Failed	—	The run stops and names accommodation as the failing specialist.

8.2 Member A

Activity, sequence and state machine diagrams from AH-A1 and A's use case specification.

8.3 Member B

Activity, sequence and state machine diagrams from AH-B1 and B's use case specification.

8.4 Member D

Activity, sequence and state machine diagrams from AH-D1 and D's use case specification.

8.5 Member E

Activity, sequence and state machine diagrams from AH-E1 and E's use case specification.

9. Relationships between key components and future development

Component	Depends on	Extension point for future work
Shared contracts (TripBrief, AgentProposal, TripPlan, ports)	nothing	A new field is added once, validated by Zod everywhere; a contract change needs an Agent Note, so every owner sees it.
Specialist interface and registry	contracts	A sixth specialist (for example, events or visas) implements <code>invoke</code> and joins the registry; the graph and board need no change beyond its budget share.
LangGraph workflow	Specialist, conflict policy, budget policy	New conflict types plug into <code>detectConflicts</code> ; the loop, round limit and scoring stay the same.
Planning board	specialist proposals	Stage order and budget shares live in one table, so a new dependency between agents is one row.
ToolGateway and ports	provider adapters	A new provider is a new adapter behind an existing port, with mock mode, typed failures and provenance kept.
MemoryStore and stores	Redis REST or process memory	Another store realises the same interface without touching agents.
Workspace UI	ChatResponse, TripPlan, NDJSON events	Reads only contracts, so new sections render through the same card and timeline components.
Agent Lab	the real workflow and specialists	New scenarios and faults are registered values, so each design change can be measured before it ships.

Planned work that these relationships support: verifying live providers end to end, an on-trip mode, flight status through a dedicated provider, and booking links through an affiliate provider.

10. Design rationale and discarded choices

Decision	Chosen	Discarded and why
Who drives the planning loop	A deterministic LangGraph state machine; models act only inside nodes	<i>Hand-written loop</i> : revision targeting and round limits were scattered. <i>Let a model drive the loop</i> : round limits, conflict detection and cost roll-up must be deterministic and testable.
How agents collaborate	A staged planning board: transport, then accommodation, then itinerary and dining, each with a budget share	<i>More rounds only</i> : every round saw the same inputs and blind budget shares overran. <i>Fully sequential</i> : destination and dining do not constrain each other, so it only adds latency. <i>Free model-to-model negotiation</i> : moves control flow into model output and cannot be tested.
Human in the loop	The traveller edits the plan in chat or the editor; sections still in conflict are marked <code>needs_you</code>	<i>Approval checkpoints</i> (the early design's "confirm key itinerary"): removed because confirming applied nothing downstream; a real checkpoint needs a booking action the product does not have.
LLM provider	DeepSeek for every model step	<i>MiniMax</i> : comparable drafts but 15 to 25 s per structured call; full pages took 37 to 81 s against 15 to 23 s. Kept wired as a backup.
Prices and places	Only from tools, with provenance on every proposal; transport and accommodation wrap deterministic calculators	<i>Let the model estimate prices</i> : invents hotels and fares.
Data mode	Mock by default, live chosen per request	<i>Deploy-time only</i> : comparing mock and live needed a redeploy. <i>Per-request environment variable</i> : leaks between concurrent serverless requests.
Server state	Redis REST store with in-process fallback	<i>Process memory</i> : lost on serverless cold starts. <i>A relational database for everything</i> : the data is small JSON by key.
Money	AUD in integer cents	Floating-point totals drift.
Evaluation	Agent Lab runs the real workflow against a single-agent baseline	<i>A separate simpler planner as baseline</i> : differences would reflect its code, not specialisation.

11. Change assessment and acceptance

Assessing a change. Every proposed change starts as a GitHub issue triaged with one of five labels (`needs-triage`, `needs-info`, `ready-for-agent`, `ready-for-human`, `wontfix`). The owner of the affected package (§5.3) assesses it against three questions:

1. Which requirement (§2) and use case (§4) does it trace to? A change with no trace is either rejected or added to the requirements first.
2. Does it touch a shared contract in `packages/shared`? If so, it needs an Agent Note in the same pull request and every package owner is told, because all packages depend on it.
3. Does it reverse an implemented decision (§10)? If so, a superseding note is written; a decision is never reversed silently.

Accepting a change. A pull request is accepted when:

- CI passes on its head: typecheck, lint, unit and workflow tests, production build, documentation link and pairing checks, and the protected-files check.
- Complex behaviour has an end-to-end test that leaves a repeatable artifact (for agent behaviour, an Agent Lab run artifact).
- The affected use case's postconditions still hold, for example `estTotal ≤ budgetTotal` or an explicit `infeasible budget` conflict for UC-C2.
- The documentation and design model describing the change are updated in the same pull request, in English and Chinese.
- The MVP definition of done still holds: grounded and labelled data, visible conflicts after edits, saved trips reopen, and provider failures degrade visibly without fictional prices.

12. Design assumptions

1. One traveller plans one trip at a time; there is no multi-user collaboration.
2. All amounts are stored and compared in AUD. A budget stated in CNY, USD or JPY is converted once, with static rates, when it is read from the traveller's message.
3. Prices are estimates for planning; the system never books or takes payment.
4. Flights are paid first, so accommodation, itinerary and dining share the money left after transport (40%, 40% and 20%).
5. Two guests share a room unless the traveller asks for individual rooms.
6. A model call can fail, time out or return off-schema output at any time, so every model step has a validated deterministic fallback.
7. Providers can be unavailable or over quota; the system then labels estimates or mock data rather than hiding the failure.
8. Three revision rounds are enough for convergence on realistic trips; beyond that the traveller decides.
9. Weather is a forecast only within 14 days; beyond that the guide gives climate context.
10. The traveller can read English or Chinese, and writes dates in any common form.

13. Contributions and use of generative AI

13.1 Individual contributions to group tasks

Fill in before submitting: the part of each diagram each member did, or a percentage. Each row adds up to 100%.

Group task	A	B	C	D	E
Project requirements and classification (§1–2)					
Feature diagram (§3)					
Overall use case diagram (§4.1)					
Architecture, package and deployment (§5)					
Class model (§6)					
Object diagram, collaboration, structured class (§7)					
Relationships, rationale, change, assumptions (§9–12)					
Video presentation					

13.2 Individual tasks

Member	Ad hoc requirement	Use case specification	Activity	Sequence	State machine
A	AH-A1	UC-A1	§8.2	§8.2	§8.2
B	AH-B1	UC-B1	§8.3	§8.3	§8.3
C	AH-C1	UC-C1, UC-C2	§8.1.1	§8.1.2	§8.1.3
D	AH-D1	UC-D1	§8.4	§8.4	§8.4
E	AH-E1	UC-E1	§8.5	§8.5	§8.5

13.3 Use of generative AI

Parts of this report were drafted with AI assistants from the group's repository and design notes, and then reviewed by the group. The models were checked against the code on `main`. Each member can explain their own diagrams in the interview.